

Sistema de Controle Bancário: NullBank

Trabalho Final – Banco de Dados – Período 2026.1

Antônio Kildere S. Menezes José Arthur G. Azevedo Daniel Neco Silva

Curso de Engenharia da Computação – Campus Sobral
Universidade Federal do Ceará

24 de Junho de 2026



Sumário

1 Introdução e Negócio

- O Modelo NullBank
- Controle de Acesso e Segurança

2 Arquitetura e Engenharia de Software

- Estrutura de Código e Segurança

3 Modelagem Relacional

- Gatilhos e Automatização (Triggers)
- Procedimentos Armazenados e Transações

4 Infraestrutura e DevOps

- Containerização e Deploy

5 Códigos-Fonte

- Visualização do Backend



Contextualização do Sistema Bancário

O **NullBank** consiste em um sistema integrado de controle e autoatendimento bancário modularizado, desenvolvido com foco em consistência relacional e segurança :

- Gestão centralizada de agências físicas e sua folha salarial.
- Cadastro unificado de funcionários, dependentes e correntistas.
- Automatização financeira rigorosa baseada em transações e gatilhos.

Para o funcionamento íntegro das regras de negócio estipuladas:

- 1 Clientes possuem o teto estrito de uma conta por agência bancária.
- 2 Contas conjuntas são limitadas a, no máximo, 2 titulares.
- 3 Funcionários podem registrar até 5 dependentes para plano de saúde.

O saldo inicial das contas é parametrizado via depósito de boas-vindas.



Níveis de Acesso Obrigatórios (Roles)

Administrador / DBA

Acesso global através das credenciais estritas Admin/Root. Possui prerrogativas totais sobre todas as entidades físicas da base de dados.

Funcionários da Agência

- **Gerente:** Cadastra funcionários, insere dependentes calculando a idade cronológica e abre contas, mas não efetua movimentações financeiras.
- **Caixa:** Acesso irrestrito a saques, depósitos e transferências na sua agência.
- **Atendente:** Permissão de leitura exclusiva sobre números e saldos locais.

Cliente

Autoatendimento restrito para checagem de extratos próprios e realização de transferências eletrônicas com verificação cruzada de CPF.



Segurança na Camada de Aplicação

Toda a base de dados de senhas foi criptografada via Hash SHA-256.

Segurança na Camada de Aplicação

Toda a base de dados de senhas foi criptografada via Hash SHA-256.

Variáveis de infraestrutura e rede são isoladas em arquivos ocultos .env



Segurança na Camada de Aplicação

Toda a base de dados de senhas foi criptografada via Hash SHA-256.

Variáveis de infraestrutura e rede são isoladas em arquivos ocultos .env

Interface desenvolvida de forma modular in Python utilizando o Streamlit.



Integridade Física no Banco de Dados

A lógica de consistência matemática reside diretamente no SGBD MySQL, evitando falhas de sincronismo originadas pela interface visual:

- `tg_funcionario_[insert/update/delete]`: Monitora a tabela de funcionários e recalcula atômicamente o montante salarial acumulado (`sal_total`) da agência vinculada.
- `tg_atualizar_saldo_transacao`: Gatilho crítico disparado **AFTER INSERT** em `Transacao`. Avalia as restrições financeiras da conta, positiva depósitos/estornos e bloqueia saques ou pagamentos indevidos.

Lançamento de Exceções Nativo

Se o saldo resultante violar os limites de crédito especial ou ficar negativado em contas comuns, o banco aborta a operação via `SIGNAL SQLSTATE '45000'`.



Mecanismo de Transferência Atômica

A rotina de movimentação eletrônica entre correntistas foi centralizada sob os pilares do conceito ACID no banco de dados:

`sp_executar_transferencia`

- Recebe as contas de origem, destino, valor financeiro e os identificadores de auditoria gerados.
- Valida nativamente a titularidade do usuário solicitante via CPF/Matrícula, blindando a transação contra manipulações externas.
- Abre um bloco transacional explícito com `START TRANSACTION`.
- Insere de forma síncrona as transações e efetua o `COMMIT`.
- Emprega um `EXIT HANDLER FOR SQLEXCEPTION` para realizar o `ROLLBACK` imediato em qualquer cenário de instabilidade de rede ou falha.



Isolamento de Ambiente e Deploy Seguro

Orquestração via Docker

Toda a aplicação encontra-se dockerizada de forma isolada. Foram estruturados containers separados para o Backend (MySQL Server 5.7) com mapeamento de volumes físicos persistentes, e para o Frontend (Streamlit em ambiente Python 3.11+).

Cloudflare Tunnel

Para a publicação estável do ambiente de homologação sem expor a infraestrutura local a vetores de ataque e dispensando configurações complexas de redirecionamento de portas em NAT, utilizou-se o **Cloudflare Tunnel**, estabelecendo conexões de saída puramente criptografadas de ponta a ponta.

Código-Fonte: Gerenciamento de Conexões

```
1      import mysql.connector
2
3      def conectar_banco():
4          config = carregar_config_banco()
5          try:
6              conexao = mysql.connector.connect(
7                  host=config["host"],
8                  user=config["user"],
9                  password=config["password"],
10                 database=config["database"]
11             )
12             return conexao
13         except mysql.connector.Error as err:
14             return None
15
```

Código-Fonte: Fluxo Principal e Roles

```
1  # Fluxo se NAO estiver logado
2  if st.session_state['usuario'] is None:
3      login.render()
4  else:
5      user = st.session_state['usuario']
6
7  # Tratamento Dinamico de Acesso baseado em Roles
8  if user['role'] == 'client':
9      abas.append(("Extrato", lambda: extrato.render(user)))
10     abas.append(("Transferir", lambda: transferencia.render(user)))
11     elif user['role'] == 'admin':
12         abas.append(("Painel DBA", lambda: gerente.render(user)))
13
```



Regras lógicas: Gatilho de Saldo (Trigger)

```
1 CREATE TRIGGER tg_atualizar_saldo_transacao AFTER INSERT ON Transacao
2 FOR EACH ROW
3 BEGIN
4 -- Verificacao se a transacao vai estourar o saldo ou limite
5 IF NEW.tipo IN ('saque', 'pagamento', 'transferencia', 'PIX') THEN
6 IF v_tipo_conta = 'conta especial' THEN
7 IF (v_saldo - NEW.valor) < (-IFNULL(v_limite, 0)) THEN
8 SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo insuficiente. Limite excedido.';
9 END IF;
10 ELSE
11 IF (v_saldo - NEW.valor) < 0 THEN
12 SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo insuficiente.';
13 END IF;
14 END IF;
15 UPDATE Conta SET saldo = saldo - NEW.valor WHERE num_conta = NEW.num_conta;
16 ELSEIF NEW.tipo IN ('deposito', 'estorno') THEN
17 UPDATE Conta SET saldo = saldo + NEW.valor WHERE num_conta = NEW.num_conta;
18 END IF;
19 END
20
```



Abstração Relacional: Visão Institucional (View)

```
1 CREATE VIEW v_contas_gerente AS
2 SELECT
3     c.matricula_gerente ,
4     c.num_conta ,
5     c.tipo_conta ,
6     c.saldo ,
7     cl.cpf ,
8     cl.nome_completo
9 FROM Conta c
10 JOIN Titularidade_Conta tc ON c.num_conta = tc.num_conta
11 JOIN Cliente cl ON tc.cpf_cliente = cl.cpf;
```



Queries Avançadas: Relatórios Periódicos (7, 30 e 365 dias)

```
1  -- Contas correntes com maior numero de transacoes nos ultimos 7 dias
2  SELECT num_conta, COUNT(id_transacao) AS total
3  FROM Transacao
4  WHERE data_hora >= NOW() - INTERVAL 7 DAY
5  AND num_conta IN (SELECT num_conta FROM Conta WHERE tipo_conta = 'conta-corrente')
6  GROUP BY num_conta
7  ORDER BY total DESC;
8
9  -- Contas com maior volume (valor total) financeiro nos ultimos 30 dias
10 SELECT num_conta, SUM(valor) AS volume
11 FROM Transacao
12 WHERE data_hora >= NOW() - INTERVAL 30 DAY
13 GROUP BY num_conta
14 ORDER BY volume DESC;
15
```



Obrigado pela Atenção!